

Part 3

(30 min)

First hands-on session:

Analyzing and comparing convergence rates
with the PEPit toolbox

Outline

Software toolboxes

Hands On

- Demo: convergence of gradient descent

- Task 1: compare three algorithms

- Task 2: examine a worst-case instance

Toolboxes

Please visit <https://github.com/PerformanceEstimation>

PerformanceEstimation

This organization regroups works/packages/toolboxes related to performance estimation problems.

If you find some content useful, please don't hesitate to give feedbacks and or to star the content!

Current packages

- [PEPit](#): allows a quick access to performance estimation problems in Python.
- [PEPit.jl](#): Julia version.
- [PESTO](#): heritage Matlab version.

There are also related works on using performance estimation problems to find Lyapunov (or potential or energy) functions. Those techniques are not yet included in PEPit and PESTO (for structural reasons), which currently only allow studying predefined Lyapunov functions (rather than finding them). Three of those works provide easy-to-use interfaces:

- historical reference (outdated matlab code): [Lyapunov functions for linear convergence of first-order methods for smooth strongly convex problems](#),
- [AutoLyap](#) (in Python).
- [AutoLyap.jl](#) (in Julia).

🔗 Education

- Informal introduction to PEPs: [here](#).
- **New** a complete tutorial on performance estimation problems with interactive labs [here](#)
- Older (shorter) tutorial with exercises: [here](#).

Events

If you organize a PEP-event, we would be happy to list your event below.

Past events:

- March 2026: [Interactive mini-course \(9h\) at SMAI-MODE](#)
- February 2023: [PEP-talks](#).

- ▶ Python toolbox **PEPit**
[Goujaud, Moucer, Glineur, Hendrickx, Taylor]
- ▶ MATLAB toolbox **PESTO**
[Taylor, Hendrickx, G]
- ▶ Julia toolbox [Das Gupta]
- ▶ Demo notebooks
- ▶ Tutorials (including this one)
- ▶ Long curated list of examples
- ▶ Events, etc.

Outline

Software toolboxes

Hands On

- Demo: convergence of gradient descent

- Task 1: compare three algorithms

- Task 2: examine a worst-case instance

Example: built-in function for smooth convex gradient descent

1.1 Gradient descent

```
PEPit.examples.unconstrained_convex_minimization.wc_gradient_descent(L, gamma, n,  
wrapper='cvxpy', solver=None, verbose=1) \[source\]
```

Consider the convex minimization problem

$$f_{\star} \triangleq \min_x f(x),$$

where f is L -smooth and convex.

This code computes a worst-case guarantee for **gradient descent** with fixed step-size γ . That is, it computes the smallest possible $\tau(n, L, \gamma)$ such that the guarantee

$$f(x_n) - f_{\star} \leq \tau(n, L, \gamma) \|x_0 - x_{\star}\|^2$$

is valid, where x_n is the output of gradient descent with fixed step-size γ , and where $x_{\star}$ is a minimizer of f .

In short, for given values of n , L , and γ , $\tau(n, L, \gamma)$ is computed as the worst-case value of $f(x_n) - f_{\star}$ when $\|x_0 - x_{\star}\|^2 \leq 1$.

Algorithm: Gradient descent is described by

$$x_{t+1} = x_t - \gamma \nabla f(x_t),$$

where γ is a step-size.

Theoretical guarantee: When $\gamma \leq \frac{1}{L}$, the **tight** theoretical guarantee can be found in [1, Theorem 3.1]:

$$f(x_n) - f_{\star} \leq \frac{L}{4nL\gamma + 2} \|x_0 - x_{\star}\|^2,$$

which is tight on some Huber loss functions.

Code: gradient method on smooth convex functions

```
# Instantiate PEP
problem = PEP()

# Declare a smooth convex function
func = problem.declare_function(SmoothConvexFunction, L=L)

# Start by defining its unique optimal point  $x_s = x_*$  and corresponding function value  $f_s = f_*$ 
xs = func.stationary_point()
fs = func(xs)

# Then define the starting point  $x_0$  of the algorithm
x0 = problem.set_initial_point()

# Set the initial constraint that is the distance between  $x_0$  and  $x^*$ 
problem.set_initial_condition((x0 - xs) ** 2 <= 1)

# Run n steps of the GD method
x = x0
for _ in range(n):
    x = x - gamma * func.gradient(x)

# Set the performance metric to the function values accuracy
problem.set_performance_metric(func(x) - fs)

# Solve the PEP
pepit_verbose = max(verbose, 0)
pepit_tau = problem.solve(wrapper=wrapper, solver=solver, verbose=pepit_verbose)

# Compute theoretical guarantee (for comparison)
theoretical_tau = L / (2 * (2 * n * L * gamma + 1))

# Print conclusion if required
if verbose != -1:
    print('*** Example file: worst-case performance of gradient descent with fixed step-sizes ***')
    print('\tPEPit guarantee:\t  $f(x_n) - f_* \leq \{:.6\} \|x_0 - x_*\|^2$ '.format(pepit_tau))
    print('\tTheoretical guarantee:\t  $f(x_n) - f_* \leq \{:.6\} \|x_0 - x_*\|^2$ '.format(theoretical_tau))

# Return the worst-case guarantee of the evaluated method (and the reference theoretical value)
return pepit_tau, theoretical_tau
```

Output: gradient method on smooth convex functions

```
>>> L = 3
>>> pepit_tau, theoretical_tau = wc_gradient_descent(L=L, gamma=1 / L, n=4, wrapper="cvxpy", solver=None, verbose=1)
(PEPit) Setting up the problem: size of the Gram matrix: 7x7
(PEPit) Setting up the problem: performance measure is the minimum of 1 element(s)
(PEPit) Setting up the problem: Adding initial conditions and general constraints ...
(PEPit) Setting up the problem: initial conditions and general constraints (1 constraint(s) added)
(PEPit) Setting up the problem: interpolation conditions for 1 function(s)
      Function 1 : Adding 30 scalar constraint(s) ...
      Function 1 : 30 scalar constraint(s) added
(PEPit) Setting up the problem: additional constraints for 0 function(s)
(PEPit) Compiling SDP
(PEPit) Calling SDP solver
(PEPit) Solver status: optimal (wrapper:cvxpy, solver: MOSEK); optimal value: 0.1666666649793714
(PEPit) Primal feasibility check:
      The solver found a Gram matrix that is positive semi-definite up to an error of 6.325029629397552e-10
      All the primal scalar constraints are verified up to an error of 6.63361402267193e-09
(PEPit) Dual feasibility check:
      The solver found a residual matrix that is positive semi-definite
      All the dual scalar values associated with inequality constraints are nonnegative up to an error of
      7.0696174975682125e-09
(PEPit) The worst-case guarantee proof is perfectly reconstituted up to an error of 7.547097551421911e-08
(PEPit) Final upper bound (dual): 0.1666666733194192 and lower bound (primal example): 0.1666666649793714
(PEPit) Duality gap: absolute: 8.340047791266514e-09 and relative: 5.004028725419552e-08
*** Example file: worst-case performance of gradient descent with fixed step-sizes ***
PEPit guarantee:       $f(x_n) - f_* \leq 0.166667 ||x_0 - x_*||^2$ 
Theoretical guarantee:  $f(x_n) - f_* \leq 0.166667 ||x_0 - x_*||^2$ 
```

Contains numerous introductory examples

(slide slightly outdated, current count is 96)

- 1. Unconstrained convex minimization

- 1.1. Gradient descent
- 1.2. Subgradient method
- 1.3. Subgradient method under restricted secant inequality and error bound
- 1.4. Gradient descent with exact line search
- 1.5. Conjugate gradient
- 1.6. Heavy Ball momentum
- 1.7. Accelerated gradient for convex objective
- 1.8. Accelerated gradient for strongly convex objective
- 1.9. Optimized gradient
- 1.10. Optimized gradient for gradient
- 1.11. Robust momentum
- 1.12. Triple momentum
- 1.13. Information theoretic exact method
- 1.14. Cyclic coordinate descent
- 1.15. Proximal point
- 1.16. Accelerated proximal point
- 1.17. Inexact gradient descent
- 1.18. Inexact gradient descent with exact line search
- 1.19. Inexact accelerated gradient
- 1.20. Epsilon-subgradient method
- 1.21. Gradient descent for quadratically upper bounded convex objective
- 1.22. Gradient descent with decreasing step sizes for quadratically upper bounded convex objective
- 1.23. Conjugate gradient for quadratically upper bounded convex objective
- 1.24. Heavy Ball momentum for quadratically upper bounded convex objective

- 2. Composite convex minimization

- 2.1. Proximal gradient
- 2.2. Accelerated proximal gradient
- 2.3. Bregman proximal point
- 2.4. Douglas Rachford splitting
- 2.5. Douglas Rachford splitting contraction
- 2.6. Accelerated Douglas Rachford splitting
- 2.7. Frank Wolfe
- 2.8. Improved interior method
- 2.9. No Lips in function value
- 2.10. No Lips in Bregman divergence
- 2.11. Three operator splitting

- 3. Non-convex optimization

- 3.1. Gradient Descent
- 3.2. No Lips 1
- 3.3. No Lips 2

- 4. Stochastic and randomized convex minimization

- 4.1. Stochastic gradient descent
- 4.2. Stochastic gradient descent in overparametrized setting
- 4.3. SAGA
- 4.4. Point SAGA
- 4.5. Randomized coordinate descent for smooth strongly convex functions
- 4.6. Randomized coordinate descent for smooth convex functions

Contains numerous introductory examples

(slide slightly outdated, current count is 96)

- 5. Monotone inclusions and variational inequalities

- 5.1. Proximal point
- 5.2. Accelerated proximal point
- 5.3. Optimal Strongly-monotone Proximal Point
- 5.4. Douglas Rachford Splitting
- 5.5. Three operator splitting
- 5.6. Optimistic gradient
- 5.7. Past extragradient

- 6. Fixed point

- 6.1. Halpern iteration
- 6.2. Optimal Contractive Halpern iteration
- 6.3. Krasnoselskii-Mann with constant step-sizes
- 6.4. Krasnoselskii-Mann with increasing step-sizes

- 7. Potential functions

- 7.1. Gradient descent Lyapunov 1
- 7.2. Gradient descent Lyapunov 2
- 7.3. Accelerated gradient method

- 8. Inexact proximal methods

- 8.1. Accelerated inexact forward backward
- 8.2. Partially inexact Douglas Rachford splitting
- 8.3. Relatively inexact proximal point

- 9. Adaptive methods

- 10. Low dimensional worst-cases scenarios

- 10.1. Inexact gradient
- 10.2. Non-convex gradient descent
- 10.3. Optimized gradient
- 10.4. Frank Wolfe
- 10.5. Proximal point
- 10.6. Halpern iteration
- 10.7. Alternate projections
- 10.8. Averaged projections
- 10.9. Dykstra

- 11. Continuous-time models

- 11.1. Gradient flow for strongly convex functions
- 11.2. Gradient flow for convex functions
- 11.3. Accelerated gradient flow for strongly convex functions
- 11.4. Accelerated gradient flow for convex functions

- 12. Tutorials

Time for some coding session!

Let us run our first PEPit example



or visit

<https://github.com/PerformanceEstimation/Tutorial-SIAM-OP26>